

INFORMAÇÕES

Módulo: 09

Aula: 05 - Hooks, Context/Provider, ReactDOM x DOM, `export function` e Yarn

Instrutor: (preencher)

Duração sugerida: 2h

CONTEÚDO

1) Um pulinho na aula anterior

Na aula anterior, trabalhamos **componentes, props e composição de interface** com React + TypeScript.

Criamos componentes reutilizáveis, entendemos tipagem de props e praticamos callbacks.

Nesta aula, vamos avançar para o próximo nível da arquitetura React:

- **Hooks no dia a dia;**
- compartilhamento de estado com **Context + Provider;**
- diferença entre **DOM e ReactDOM;**
- padrão de exportação com `export function`;
- uso de **Yarn** no fluxo de desenvolvimento.

Importante: **não** entraremos em `useMemo` nesta aula. Esse tema ficará para a próxima.

2) O que veremos hoje?

Vamos trabalhar cinco pilares:

1. **Hooks essenciais** (`useState` , `useEffect` , `useContext`)
 2. `export function` (named export) vs `export default`
 3. **Context + Provider** para evitar prop drilling excessivo
 4. **ReactDOM x DOM**
 5. **Yarn no dia a dia** (`yarn install` , `yarn dev` , `yarn build` , lockfile)
-

3) Hooks: o que são e por que usar

Hooks são funções do React que permitem usar recursos como estado, ciclo de vida e contexto em componentes funcionais.

Hooks foco da aula

- `useState` : guarda estado local
- `useEffect` : executa efeitos colaterais
- `useContext` : consome dados compartilhados de um contexto

Exemplo rápido

```
import { useEffect, useState } from "react";

export function Contador(): JSX.Element {
  const [valor, setValor] = useState<number>(0);

  useEffect(() => {
    document.title = `Valor: ${valor}`;
  }, [valor]);

  return (
    <div>
      <p>Valor atual: {valor}</p>
      <button onClick={() => setValor((v) => v + 1)}>Somar</button>
    </div>
  );
}
```

4) ReactDOM x DOM (sem confusão)

DOM

É a estrutura da página no navegador (árvore de elementos HTML).

ReactDOM

É a biblioteca que conecta React ao navegador, renderizando componentes React dentro do DOM real.

Em termos práticos:

- **DOM** = estrutura real da interface no browser
- **ReactDOM** = ponte que aplica as atualizações feitas pelo React nessa estrutura

Exemplo comum em `main.tsx`:

```
import ReactDOM from "react-dom/client";
import App from "./App";

ReactDOM.createRoot(document.getElementById("root")!).render(<App />);
```

5) export function vs export default

Named export (recomendado nesta aula)

```
export function UserCard(): JSX.Element {
  return <div>Card do usuario</div>;
}
```

Importação:

```
import { UserCard } from "./UserCard";
```

Default export

```
export default function UserCard(): JSX.Element {
  return <div>Card do usuario</div>;
}
```

Importação:

```
import UserCard from "./UserCard";
```

Regra prática para o time

- Se o projeto padroniza named export, preferir `export function`.
 - Isso facilita import explícito e refatoração.
-

6) Context e Provider

Quando muitos componentes precisam do mesmo dado (tema, usuário logado, permissões, etc.), passar props por vários níveis pode ficar ruim.

O padrão **Context + Provider** resolve isso de forma organizada.

Estrutura mínima

```
import { createContext, useContext, useState } from "react";

type Tema = "claro" | "escuro";

type TemaContextType = {
  tema: Tema;
  alternarTema: () => void;
};

const TemaContext = createContext<TemaContextType | null>(null);

export function TemaProvider({ children }: { children: React.ReactNode }) {
  const [tema, setTema] = useState<Tema>("claro");

  function alternarTema(): void {
    setTema((anterior) => (anterior === "claro" ? "escuro" : "claro"));
  }

  return (
    <TemaContext.Provider value={Undefined, Undefined}>
      {children}
    </TemaContext.Provider>
  );
}

export function useTema(): TemaContextType {
  const contexto = useContext(TemaContext);
  if (!contexto) {
    throw new Error("useTema deve ser usado dentro de TemaProvider");
  }
}
```

```
    }  
    return contexto;  
  }  
}
```

Uso:

```
export function BotaoTema(): JSX.Element {  
  const { tema, alternarTema } = useTema();  
  
  return (  
    <button onClick={alternarTema}>  
      Tema atual: {tema}  
    </button>  
  );  
}
```

7) Yarn no fluxo do projeto

Mesmo que alguns exemplos usem npm, nesta trilha adotaremos **Yarn** como referência principal.

Comandos essenciais

- Instalar dependências:

```
yarn install
```

- Rodar ambiente de desenvolvimento:

```
yarn dev
```

- Gerar build:

```
yarn build
```

- Rodar preview do build:

```
yarn preview
```

Sobre o `yarn.lock`

- Garante versões reproduzíveis entre máquinas e CI.
 - Deve ser versionado no Git.
 - Não deve ser editado manualmente.
-

8) Hands-on da aula

Objetivo do hands-on

Criar uma tela simples que:

1. Usa `useState` para estado local;
2. Usa Context + Provider para compartilhar tema;
3. Usa `useEffect` para atualizar o título da página;
4. Usa `export function` nos novos componentes;
5. É executada pelo **Yarn**.

Estrutura sugerida

```
src/  
  context/  
    TemaContext.tsx  
  components/  
    BotaoTema.tsx  
    PainelInfo.tsx  
  App.tsx  
  main.tsx
```

Passos

1. Criar `TemaContext.tsx` com Provider e hook `useTema`.
 2. Envolver `App` com `TemaProvider`.
 3. Criar `BotaoTema` para alternar tema.
 4. Criar `PainelInfo` com contador local e exibição do tema atual.
 5. Em `App`, usar `useEffect` para atualizar `document.title` com tema e contador.
 6. Rodar `yarn dev` e validar comportamento.
-

9) Erros comuns desta aula

1. Usar `useContext` fora do Provider

Resultado: erro em runtime.

2. Esquecer dependências do `useEffect`

Resultado: efeito com valores desatualizados ou execução fora do esperado.

3. Misturar padrão de export sem critério

Resultado: imports inconsistentes e manutenção mais difícil.

4. Alterar lockfile manualmente

Resultado: inconsistência de dependências.

10) Exercícios de fixação

1. O que o `useContext` faz?

- ☐ Cria estilos CSS para o componente
- ☒ Permite consumir valores compartilhados por um Context
- ☐ Substitui totalmente o `useState`
- ☐ Executa chamadas HTTP automaticamente

2. Qual é a função do ReactDOM?

- ☐ Criar APIs REST
- ☐ Compilar TypeScript
- ☒ Conectar a aplicação React ao DOM do navegador
- ☐ Substituir o Vite

3. Em qual cenário o Context + Provider faz mais sentido?

- ☐ Quando só um componente usa o dado
- ☒ Quando vários níveis da árvore precisam do mesmo estado
- ☐ Quando queremos evitar TypeScript
- ☐ Quando o projeto não usa hooks

4. Qual comando instala dependências com Yarn?

- ☐ yarn start
- ☒ yarn install
- ☐ yarn build
- ☐ yarn preview

5. O que representa `export function Componente()` ?

- (X) Exportação nomeada do componente
 - () Exportação default obrigatória
 - () Um hook customizado
 - () Um contexto global
-

11) Desafio para casa

Criar uma mini tela com:

- um `Provider` de preferência de exibição (`compacto / confortavel`);
- um botão que alterna a preferência;
- um bloco de conteúdo que consome essa preferência via `useContext` ;
- um contador local com `useState` ;
- `useEffect` atualizando `document.title` .

Regras

- usar `export function` nos novos componentes;
 - tipar corretamente props e contexto em TypeScript;
 - rodar com `yarn dev` ;
 - manter código organizado por responsabilidade.
-

12) Resumo final

Ao final desta aula, o aluno deve conseguir:

- explicar a diferença entre **DOM** e **ReactDOM**;
- usar `useState` , `useEffect` e `useContext` em cenários reais;
- criar e consumir um **Context + Provider**;
- aplicar padrão de exportação com `export function` ;
- executar e validar o projeto com **Yarn**.

Na próxima aula, avançaremos para `useMemo` com foco em quando realmente vale usar.